Database Management System

1. What is Database Management System (DBMS)?

   Database Management System (DBMS) is software that creates, maintains, and provides access to a database, acting as an interface between users and the data itself. It handles tasks like storing, organizing, retrieving, and updating data, while also ensuring data integrity, security, and controlled access for multiple users simultaneously.

2. What are the Key Functions of Database Management System (DBMS)?

   Data Definition: A DBMS defines the structure of the database, including the format, names, and records of the data.
   Data Manipulation: It allows users to create, read, update, and delete data within the database.
   Data Storage Management: It manages the physical storage of data on a computer system.
   Security: A DBMS enforces security rules to control who can access and modify the data.
   Data Integrity: It implements rules to ensure the data is accurate, consistent, and reliable.
   Concurrency Control: It manages simultaneous access by multiple users, preventing conflicts and data corruption.
   Backup and Recovery: It provides mechanisms for backing up data and restoring it in case of system failures.
   Administration: It supports administrative tasks such as performance monitoring, tuning, and auditing.

3. Why are Database Management Systems important?

   Organization and Efficiency: DBMSs provide an organized and efficient way to store and access large amounts of data.
   Data Consistency: By defining and enforcing rules, they help maintain consistent data across the entire database.
   Data Security: They offer robust security features to protect sensitive information.
   User-Friendliness: They serve as an interface, simplifying complex data operations for users and applications.
   Scalability: They are designed to manage large and growing datasets for businesses of all sizes.

4. Explain various types of Database Management Systems

   a) Hierarchical DBMS
      a. Stores data in a tree-like structure with parent-child relationships.
      b. Each parent can have multiple children, but a child can have only one parent.
      c. Data navigation is done through paths.
      d. E.g. IMS (Information Management System)

b) Graph DBMS
    a. Stores data in graph structures with nodes (entities) and edges (relationships).
    b. Uses graph query languages like Gremlin or SPARQL.
    c. E.g: Neo4j, Amazon Neptune

c) Relational DBMS (RDBMS)
    a. Stores data in tables with rows and columns.
    b. Relationships are established using keys.
    c. Data access is done using SQL (Structured Query Language).
    d. E.g: Oracle, SQL Server

d) Document Database
    a. Stores data in flexible, semi-structured documents (e.g., JSON, XML, BSON).
    b. No predefined schema, allowing for dynamic data structures.
    c. Data access is done using query languages specific to the document database
    d. E.g: MongoDB

e) Timeseries Database (TSDB)
    a. Built specifically for handling metrics and events or measurements that are time-stamped.
    b. Optimized for measuring change over time.
    c. E.g: Prometheus, Amazon Timestream

f) Key-Value Database
    a. Stores data as a collection of key-value pairs
    b. Provides high performance and scalability
    c. E.g: Amazon DynamoDB

g) Other databases
    a. Network Database
    b. Object Oriented Database
    c. Vector Database

5. What are the differences between RDBMS and NOSQL Databases?

Feature	**RDBMS**	**NoSQL**
Data Model	Structured tables with fixed schema	Flexible and schema-less (document, key-value, etc.)
Query Language	SQL	Varies (often proprietary APIs/languages)
Data Integrity	Strong ACID compliance	Varies, often prioritizes availability
Scalability	Primarily vertical	Horizontal

Use Cases	• Structured • Transactional data	• Unstructured/semi-structured data • Big data • Real-time apps

6. Explain the ACID properties of RDBMS.

   In a RDBMS, transaction is a single logical operation on the data. Generally, transactions are composed of multiple statements of work. ACID (Atomicity, Consistency, Isolation, Durability) are the properties of the transaction as detailed below.

   Atomicity: Guarantees that each transaction either succeeds completely or fails completely. This is achieved by rolling back the statements of work even if one of them fails.

   Consistency: Ensures preservation of database invariants such as referential integrity i.e. all defined rules are adhered during data writing.

   Isolation: Transactions are often executed concurrently (e.g., multiple transactions reading and writing to a table at the same time). Isolation ensures that concurrent execution of transactions provides the same result as if the transactions are executed sequentially.

   Durability: Ensures update due to a transaction is not lost even after system failure.

7. Explain important Keys of RDBMS

   Candidate Key: Any attribute or set of attributes that can uniquely identify a record (row) in a table. It should satisfy below properties

   Uniqueness: The values in the candidate key attributes must be unique for every row in the table.
   Minimality: No proper subset of the candidate key can also uniquely identify the rows. If a smaller set of attributes within the candidate key can still uniquely identify the rows, then the larger set is not minimal.

   Note: A table can have multiple candidate keys.

   Primary Key: A candidate key chosen to uniquely identify the rows in a table. A table can have only 1 Primary Key.

   Composite Key: Two or more attributes when combined can uniquely identify the rows in a table.

   Foreign Key: An attribute that is the primary key in another table. It is useful for drawing relations between the tables.

Natural Key: An attribute that can uniquely identify a row, and exists in the real world

Surrogate Key: An attribute which doesn't exist in real world but added to uniquely identify a row

Super Key: Any combination of attributes that can uniquely identify a record (row) in a table.

Unique Key: Any combination of attributes where the value cannot be repeated for any other record (row) in a table.

Not Null: The attribute value cannot be NULL for any record (row) in a table.

Check: The condition/criteria that should be met during a record (row) creation or update.

Default: The value assigned to an attribute when no specific value is provided during record (row) creation.

8. Explain Normalization

   Normalization refers to organizing data in a database by reducing redundancy and improving data integrity.

   Purpose: Aims to eliminate redundant data and improve data integrity by organizing data into tables and defining relationships between them according to standard rules called normal forms.
   How it Works: Typically involves breaking down large tables into smaller, more manageable ones, which reduces issues like data inconsistency that can occur during insertion, deletion, or update operations.
   Benefits: Leads to more efficient, accurate, and scalable databases that are easier to manage and adapt to changing business needs.

9. Explain Denormalization

   Denormalization refers to increasing redundancy in a database. This is done primarily to improve the performance. Reporting systems generally hold the data in denormalized form. E.g. Start Schema.

10. Explain Normal Forms

    a) First Normal Form (1NF)
       a. Goal: A Table with no repeating groups of data and values of each column are atomic.
       b. Key Rules:
          i. Each column must contain only a single, atomic value (no multi-valued cells).
          ii. Each row must be unique and identifiable by a primary key.

    b) Second Normal Form (2NF)
       a. Goal: Reduce redundancy by removing partial dependencies.

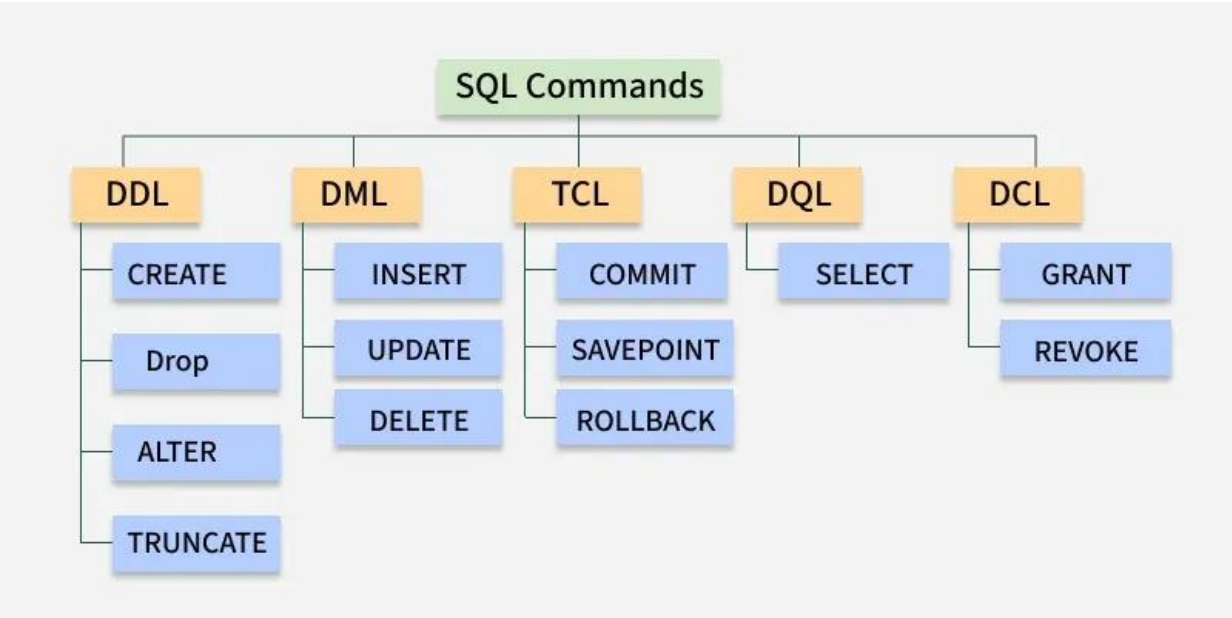
b. Key Rules:
   i. The table must already be in 1NF.
   ii. All non-key attributes (attributes not part of the primary key) must be fully functionally dependent on the entire primary key, not just a part of it.

c) Third Normal Form (3NF)
   a. Goal: Eliminate data duplication by removing transitive dependencies.
   b. Key Rules:
      i. The table must be in 2NF.
      ii. There must be no transitive dependencies i.e. no non-key attribute should be dependent on another non-key attribute.

d) Other Normal Forms – BCNF, 4NF, PJNF (5NF), DKNF (6NF)

11. Provide Overview of SQL

Structured Query Language (SQL) is the standard language used to interact with relational databases.



12. List the Reserved words of MySQL

ACCESSIBLE	ADD	ALL
ALTER	ANALYZE	AND
AS	ASC	ASENSITIVE
BEFORE	BETWEEN	BIGINT

BINARY	BLOB	BOTH
BY	CALL	CASCADE
CASE	CHANGE	CHAR
CHARACTER	CHECK	COLLATE
COLUMN	CONDITION	CONSTRAINT
CONTINUE	CONVERT	CREATE
CROSS	CURRENT_DATE	CURRENT_TIME
CURRENT_TIMESTAMP	CURRENT_USER	CURSOR
DATABASE	DATABASES	DAY_HOUR
DAY_MICROSECOND	DAY_MINUTE	DAY_SECOND
DEC	DECIMAL	DECLARE
DEFAULT	DELAYED	DELETE
DESC	DESCRIBE	DETERMINISTIC
DISTINCT	DISTINCTROW	DIV
DOUBLE	DROP	DUAL
EACH	ELSE	ELSEIF
ENCLOSED	ESCAPED	EXISTS
EXIT	EXPLAIN	FALSE
FETCH	FLOAT	FLOAT4
FLOAT8	FOR	FORCE
FOREIGN	FROM	FULLTEXT
GENERAL	GET	GRANT
GROUP	HAVING	HIGH_PRIORITY
HOUR_MICROSECOND	HOUR_MINUTE	HOUR_SECOND
IF	IGNORE	IGNORE_SERVER_IDS
IN	INDEX	INFILE
INNER	INOUT	INSENSITIVE

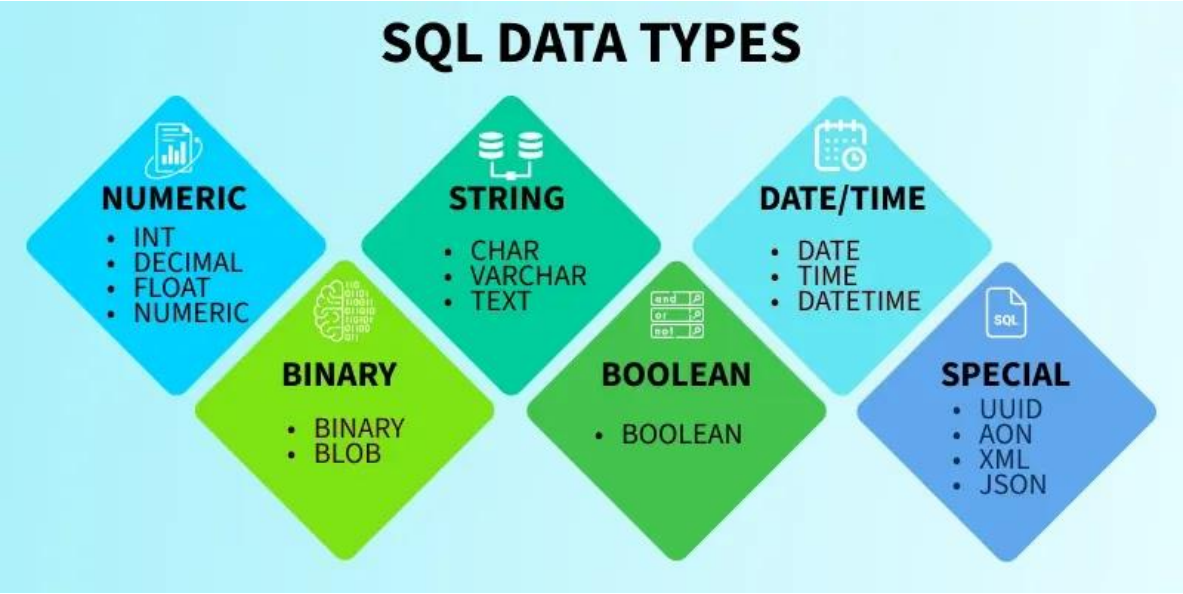
INSERT	INT	INT1
INT2	INT3	INT4
INT8	INTEGER	INTERVAL
INTO	IO_AFTER_GTIDS	IO_BEFORE_GTIDS
IS	ITERATE	JOIN
KEY	KEYS	KILL
LEADING	LEAVE	LEFT
LIKE	LIMIT	LINEAR
LINES	LOAD	LOCALTIME
LOCALTIMESTAMP	LOCK	LONG
LONGBLOB	LONGTEXT	LOOP
LOW_PRIORITY	MASTER_BIND	MASTER_HEARTBEAT_PERIOD
MASTER_SSL_VERIFY_SERVER_CERT	MATCH	MAXVALUE
MEDIUMBLOB	MEDIUMINT	MEDIUMTEXT
MIDDLEINT	MINUTE_MICROSECOND	MINUTE_SECOND
MOD	MODIFIES	NATURAL
NOT	NO_WRITE_TO_BINLOG	NULL
NUMERIC	ON	ONE_SHOT
OPTIMIZE	OPTION	OPTIONALLY
OR	ORDER	OUT
OUTER	OUTFILE	PARTITION
PRECISION	PRIMARY	PROCEDURE
PURGE	RANGE	READ
READS	READ_WRITE	REAL
REFERENCES	REGEXP	RELEASE

RENAME	REPEAT	REPLACE
REQUIRE	RESIGNAL	RESTRICT
RETURN	REVOKE	RIGHT
RLIKE	SCHEMA	SCHEMAS
SECOND_MICROSECOND	SELECT	SENSITIVE
SEPARATOR	SET	SHOW
SIGNAL	SLOW	SMALLINT
SPATIAL	SPECIFIC	SQL
SQLEXCEPTION	SQLSTATE	SQLWARNING
SQL_AFTER_GTIDS	SQL_BEFORE_GTIDS	SQL_BIG_RESULT
SQL_CALC_FOUND_ROWS	SQL_SMALL_RESULT	SSL
STARTING	STRAIGHT_JOIN	TABLE
TERMINATED	THEN	TINYBLOB
TINYINT	TINYTEXT	TO
TRAILING	TRIGGER	TRUE
UNDO	UNION	UNIQUE
UNLOCK	UNSIGNED	UPDATE
USAGE	USE	USING
UTC_DATE	UTC_TIME	UTC_TIMESTAMP
VALUES	VARBINARY	VARCHAR
VARCHARACTER	VARYING	WHEN
WHERE	WHILE	WITH
WRITE	XOR	YEAR_MONTH
ZEROFILL		

13. Provide Overview of datatypes in SQL

Data type defines the kind of data it can store, such as integers, dates, text, or binary values. Choosing the correct data type is crucial for data integrity, query performance and efficient indexing.

The various sql datatypes are provided below:



Details of sql data types can be referenced here.

14. Explain the differences between Materialized Views and Non-Materialized Views

Non-Materialized Views are generally referred as Views. A View is a virtual table which dynamically fetches data from the underlying tables whenever accessed, without storing the data physically in the database.

A Materialized View is a physical table which stores the result of the query. The view is refreshed i.e. underlying table is updated regularly either manually or automatically.

Feature	View	Materialized View
Definition	A view is a virtual table created from a query, and it doesn't store data physically.	A materialized view stores the results of a query physically in the database for faster retrieval.
Data Storage	Only the query expression is stored; the result set is generated dynamically when the view is accessed.	Query results are stored physically in the database, consuming additional storage space.

Feature	View	Materialized View
Performance	Slower for complex queries since the result set is computed dynamically on each access.	Faster as results are precomputed and stored, reducing computation time.
Update Behaviour	Automatically reflects changes in the underlying tables since data is retrieved dynamically.	Needs manual or automatic refresh to update the stored data when underlying tables change.
Storage Cost	No additional storage cost since data is not physically stored.	Requires extra storage as it saves query results.
Maintenance Cost	No maintenance cost, as views are dynamically updated with no stored data.	Involves maintenance cost due to periodic refreshes to keep data synchronized with base tables.
SQL Standards	Fully standardized and supported by all major database systems.	Not fully standardized; support and implementation vary across database systems.
Use Cases	Best for scenarios where data is accessed infrequently and requires up-to-date values.	Ideal for frequently accessed data where performance is critical, such as reporting and analytics.

15. Describe Indexes
16. Describe Relationships
17. Describe JOINs
18. Describe Subqueries
19. Describe Stored Procedures
20. Describe Functions
21. Describe Triggers
22. Describe Cursors
23. Describe Transaction Isolation Levels

    The transaction isolation levels in DBMS determine how the concurrently running transactions behave. Isolation levels define the degree to which a transaction must be isolated from the data modifications made by any other transaction in the database system.

A transaction isolation level is defined by the following phenomena

Dirty Read - A Dirty read is a situation when a transaction reads data that has not yet been committed.

Non Repeatable Read - Non-repeatable read occurs when a transaction reads the same row twice or more and does not get a same value each time

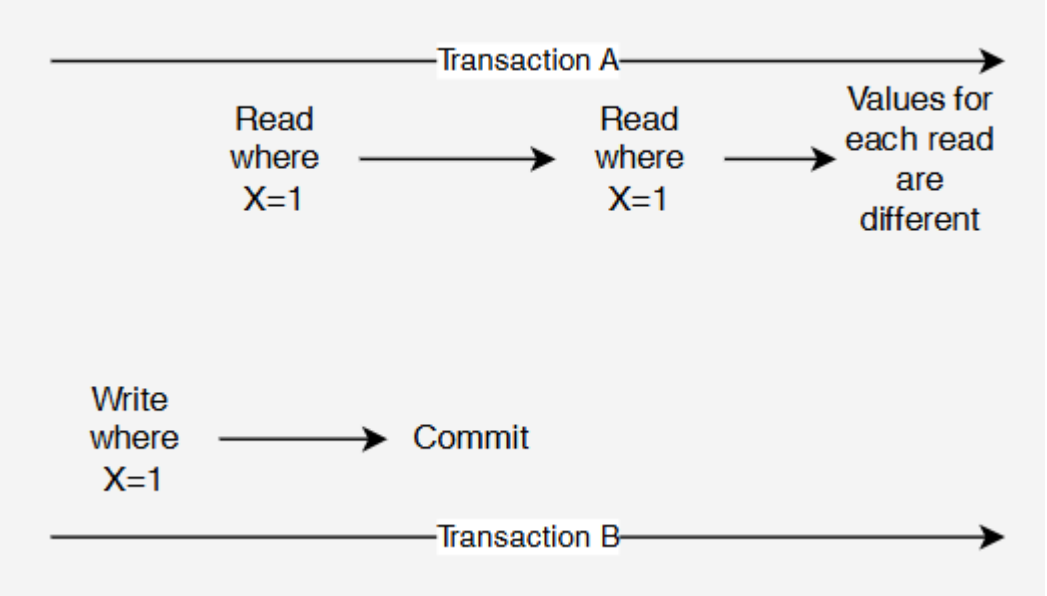
Phantom Read - Phantom Read occurs when a transaction reads using same query two or more times and does not retrieve same number of rows each time.

The SQL standard defines four isolation levels to handle these phenomena.

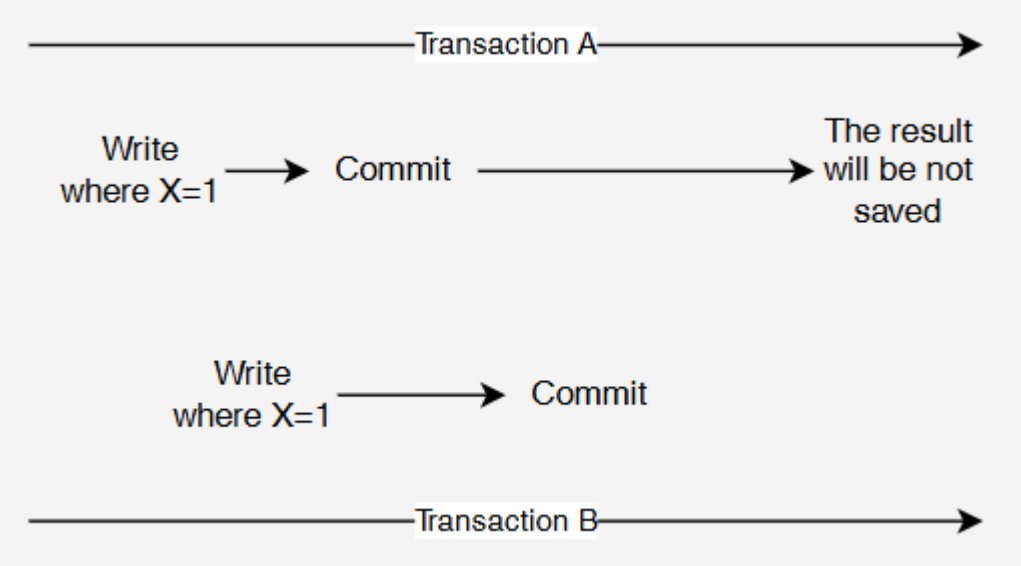
Read Uncommitted: One transaction may read changes uncommitted made by other transactions. This is the lowest isolation level and transactions are not isolated from each other. This allows dirty reads.

Read Committed: A transaction can only read changes made by other committed transactions. The transaction holds read locks while referencing the rows. However, if an update is made the lock is held till the change is committed.

It doesn't prevent writes by other transactions and can result in non-repeatable read as depicted below.

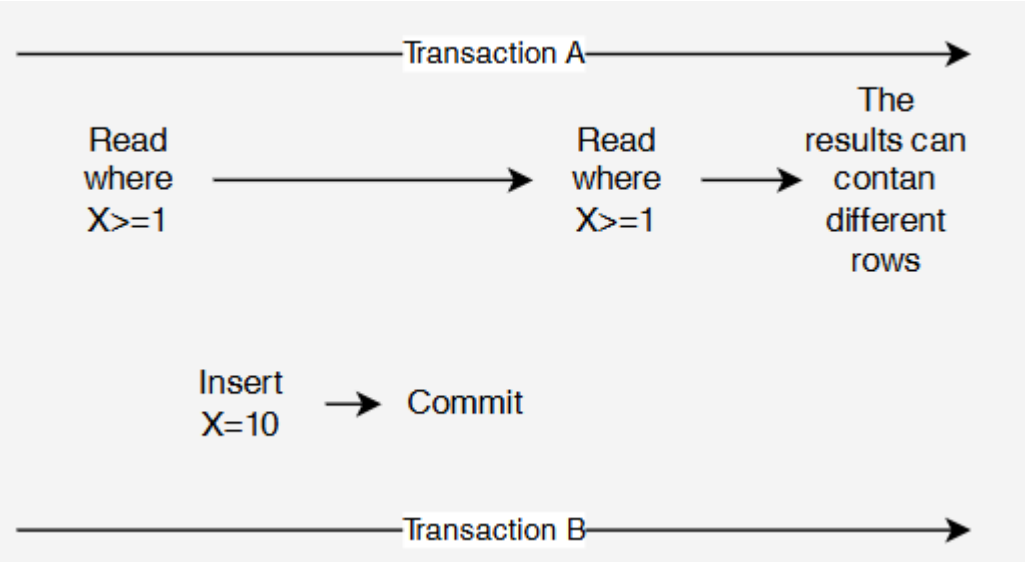


This can also result in a lost update problem i.e. one transaction overwrites the changes made by another transaction, leading to the loss of data as depicted below.



Repeatable Read - The transaction will see the same data throughout its duration, even if other transactions commit changes to the data. The transaction holds read and write locks on all referenced rows.

This doesn't prevent writing of additional rows by other transactions and can result in phantom reads as depicted below



Serializable - The transactions appear to be executing serially. This ensures that there are no dirty reads, non-repeatable reads, or phantom reads.

Isolation Level	Dirty reads	Non-repeatable reads	Phantoms
Read Uncommitted	May occur	May occur	May occur
Read Committed	Don't occur	May occur	May occur
Repeatable Read	Don't occur	Don't occur	May occur
Serializable	Don't occur	Don't occur	Don't occur

Note: All databases don't support all the isolation levels. The databases can name these isolation levels differently also. E.g: IBM DB2 calls these isolation levels as Uncommitted Read (UR), Cursor Stability (CS), Read Stability (RS), Repeatable Read (RR)

Worksheet-1

1. **Design the database with below information**

   You have a table tracking customer purchases, where
   - "Purchased Product List" contains a comma-separated list of products
   - "Customer" contains full name of customer and customer's address
   - "Product Supplier Information" contains details of supplier name and GST details

   Some of the below details are used currently
   - Supplier ID – Uniquely identifies a supplier
   - Product ID – Uniquely identifies a product
   - Order ID – Uniquely identifies each item ordered in the Purchased Product List
   - Customer ID – Uniquely identifies a customer

2. **Identify current Normal Form and Normalize Ship table to 3 NF, if required.**

   Attributes/Columns**:** BoatId, NbrOfPools, YearMade, Tonnage, CompanyId, CompanyName

   Functional Dependencies:

   - BoatId → NbrOfPools, YearMade, Tonnage, CompanyId, CompanyName
   - CompanyId → CompanyName
   - CompanyName → CompanyId

3. **Identify current Normal Form and Normalize Inventory table to 3 NF, if required.**
   Attributes/Columns**:** PartNbr, WarehouseNbr, WarehouseLocation, QOH (Quantity On Hand), Weight, PartColor

   Functional Dependencies:

   - PartNbr → Weight, PartColor
   - PartNbr + WarehouseNbr → QOH
   - WarehouseNbr → WarehouseLocation

Information Classification - Non-business

Worksheet-2

1. Complete the Exercises available at this location.